TrustScaleAI Companion — Setup Guide

Next.js 14 · Prisma · Auth.js · PostgreSQL · Railway

QualiZeal

2026-04-24

Contents

TrustScaleAI Companion — Deployable Application

A Next.js 14 + Prisma + Auth.js reference implementation that wires **real** Microsoft Entra ID / Google SSO and **real** Engagement + Stakeholder creation flows onto the TrustScaleAI product mockup. All other product surfaces are served from the static mockup (`public/mockup.html`).

This guide assumes you are deploying with your **personal free accounts**:

- `vbraju@gmail.com` (for Google OAuth)
- `vbraju@hotmail.com` (for Microsoft Entra ID)

The setup works identically with any Gmail or Hotmail/Outlook account — substitute your own where these appear in the steps.

What is real vs. mocked

Feature	Real (DB-backed)	Mocked (static HTML)
Sign-in with Microsoft (Entra ID / personal MSA)	Yes	—
Sign-in with Google (Workspace / free Gmail)	Yes	—
Dashboard — list your engagements	Yes — reads from Postgres	—
Create engagement	Yes — Server Action + Postgres	—
Add stakeholder to engagement	Yes — Server Action + Postgres	—
View engagement detail + stakeholders table	Yes	—
REST API for engagements + stakeholders	Yes	—
Full product UI (admin, templates, gates, QA, readiness, value waterfall, etc.)	—	`/mockup.html`

Tech stack

- **Next.js 14** — App Router, React Server Components, Server Actions
- **TypeScript 5**
- **Prisma 5** with **PostgreSQL** (Railway Postgres plugin)
- **Auth.js v5** (NextAuth) with `PrismaAdapter`
- **MUI 5** theme colour-matched to the mockup
- **Zod** for runtime input validation
- **Railway** for hosting, with Nixpacks builds and automatic Postgres provisioning

Total end-to-end setup time: about 30–45 minutes the first time.

---

1. Prerequisites

Before you start, make sure you have:

1. **Node.js 20 or newer** — download from https://nodejs.org. Verify with `node -v` (must print `v20.x` or higher).
2. **Git** — https://git-scm.com. Verify with `git --version`.
3. A **GitHub account** — https://github.com (free tier is fine).
4. A **Railway account** — https://railway.app (sign in with GitHub; free trial gives $5 of compute, enough for small workloads).
5. Your **free Gmail account** `vbraju@gmail.com` — no paid subscription needed.
6. Your **free Hotmail account** `vbraju@hotmail.com` — no paid Microsoft 365 subscription needed.

You do **not** need a Google Workspace subscription, an Azure subscription, or any paid Microsoft tenant. Google Cloud Console and Microsoft Entra are both free to use for OAuth app registration.

---

2. Clone and install locally (optional but recommended)

Even if you plan to deploy straight to Railway, doing one local run catches configuration mistakes early.

```
# 1. Unzip the package (or git clone the repo if you've already pushed it)
unzip trustscaleai-app.zip -d trustscaleai-app
cd trustscaleai-app

# 2. Copy the env template
cp .env.example .env

# 3. Install dependencies (takes 1–2 minutes)
npm install
```

Do not start the dev server yet — you still need OAuth credentials and a database.

---

3. Create the Google OAuth client (using vbraju@gmail.com)

3.1 Sign in to Google Cloud Console

1. Open https://console.cloud.google.com in a browser where you are signed in as `vbraju@gmail.com`.
2. If this is your first visit, accept the Google Cloud terms and country.

3.2 Create a Google Cloud project

1. Click the project dropdown at the top of the Console (next to the Google Cloud logo) → **New Project**.
2. Name: `TrustScaleAI Companion`.
3. Organization: leave as **No organization** (this is a personal Gmail, so you do not have a Workspace organization).
4. Click **Create** and wait ~20 seconds for the project to be provisioned.
5. Confirm the project dropdown now shows `TrustScaleAI Companion` as the active project.

3.3 Configure the OAuth consent screen

Because `vbraju@gmail.com` is a personal Gmail (not part of a Workspace), you must pick **External** on the consent screen.

1. Left sidebar → **APIs & Services** → **OAuth consent screen**.
2. User type → **External** → **Create**.
3. App information:
   - App name: `TrustScaleAI Companion`
   - User support email: `vbraju@gmail.com`
   - App logo: optional, skip for now

4. App domain section — leave the three fields empty; fill them in after you have a Railway URL.
5. Developer contact information: `vbraju@gmail.com`
6. **Save and Continue**.
7. **Scopes** page → click **Add or Remove Scopes** → tick only the three non-sensitive ones:
   - `openid`
   - `.../auth/userinfo.email`
   - `.../auth/userinfo.profile`

   These are the only scopes Auth.js needs. Click **Update**, then **Save and Continue**.
8. **Test users** page → click **+ Add Users** → enter `vbraju@gmail.com` (and any other Gmail addresses you want to let sign in during testing). Google allows up to 100 test users while the app is in Testing mode. Click **Save and Continue**.
9. **Summary** page → **Back to Dashboard**.

Your consent screen will say **Publishing status: Testing**. That is fine — you do not need to publish for the mockup. Publishing is only required if you want strangers (non-test-users) to sign in; for a demo, staying in Testing mode is easier and avoids Google's verification review.

3.4 Create the OAuth Web Client

1. Left sidebar → **APIs & Services** → **Credentials**.
2. Click **+ Create Credentials** → **OAuth client ID**.
3. Application type → **Web application**.
4. Name → `TrustScaleAI Web Client`.
5. **Authorized JavaScript origins** — click **+ Add URI** and add two:
   - `http://localhost:3000`
   - `https://YOUR-RAILWAY-DOMAIN` (replace after Section 6.4; you can add it later)
6. **Authorized redirect URIs** — click **+ Add URI** and add two:
   - `http://localhost:3000/api/auth/callback/google`
   - `https://YOUR-RAILWAY-DOMAIN/api/auth/callback/google`
7. **Create**.
8. A dialog shows your **Client ID** and **Client Secret**. Click **Download JSON** and save it, or copy both values into a scratch file now. You will paste them into `.env` and Railway variables.

   Important: if you only add the localhost redirect URI now, remember to come back after Section 6.4 and add the Railway callback — otherwise production sign-ins will fail with `redirect_uri_mismatch`.

---

4. Create the Microsoft Entra ID app (using vbraju@hotmail.com)

Microsoft Entra (formerly Azure AD) is Microsoft's identity platform. The catch with a personal Hotmail account is: you do not have an Entra tenant yet, but Microsoft provisions a free one for you as soon as you sign into Entra admin for the first time. No paid subscription, no credit card.

4.1 Get your free Entra tenant

1. Open a new browser window (InPrivate / Incognito is cleanest).
2. Go to https://entra.microsoft.com.
3. Sign in as `vbraju@hotmail.com`.

4. Microsoft will notice that you do not belong to an Entra tenant and will prompt you to create one. Accept the defaults (your tenant will get a name like `VbrajuHotmail.onmicrosoft.com`). Wait ~1 minute.
5. You should now be looking at the **Microsoft Entra admin center** with `vbraju@hotmail.com` in the top right and your new tenant name below it.

4.2 Register the application

1. Left sidebar → **Applications** → **App registrations**.
2. Click **+ New registration**.
3. Name: `TrustScaleAI Companion`.
4. **Supported account types** — this is the critical choice. Select:

   Accounts in any organizational directory (Any Microsoft Entra ID tenant – Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)

   This option is what lets `vbraju@hotmail.com` and other Hotmail/Outlook.com/Live.com accounts sign in. If you pick a narrower option, personal accounts will get error `AADSTS50020` when they try to sign in.
5. **Redirect URI**:
   - Platform: **Web**
   - URL: `http://localhost:3000/api/auth/callback/microsoft-entra-id`
6. Click **Register**.
7. You should now see the app overview. Copy and save these two values — you will paste them into env vars:
   - **Application (client) ID** — a GUID
   - **Directory (tenant) ID** — you will **not** use this in env vars (we use `common` instead, see 4.5) but save it for reference

4.3 Create a client secret

1. Left sidebar of the app registration → **Certificates & secrets** → **Client secrets** tab → **+ New client secret**.
2. Description: `TrustScaleAI Railway`.
3. Expires: **24 months** (or **Custom** — up to 24 months is the Microsoft limit).
4. **Add**.
5. **Copy the Value field immediately** — this is the client secret. Microsoft only shows it once; if you navigate away, it becomes a row of dots forever. Paste it into your scratch file.

   Do **not** use the "Secret ID" column — that is an internal identifier, not the secret. You need the column labelled **Value**.

4.4 Confirm the authentication settings

1. Left sidebar of the app registration → **Authentication**.
2. Under **Supported account types**, verify it reads **"Accounts in any organizational directory (Any Microsoft Entra ID tenant – Multitenant) and personal Microsoft accounts (e.g. Skype, Xbox)"**. If it does not, change it now.

3. Under **Platform configurations → Web → Redirect URIs**, you should see `http://localhost:3000/api/auth/callback/microsoft-entra-id`. Leave it; you will add the Railway URL in step 4.6.
4. Under **Implicit grant and hybrid flows**, leave both checkboxes **unchecked** (Auth.js uses the authorization code flow, which is the modern default).
5. **Save** if you changed anything.

4.5 Why `AZURE_AD_TENANT_ID=common` matters

The env var `AZURE_AD_TENANT_ID` tells the app which Microsoft identity endpoint to route sign-ins through:

- `common` — accepts both work/school accounts (any Entra tenant) **and** personal Microsoft accounts (Hotmail/Outlook.com/Live.com).
- `<your-tenant-guid>` — only accepts users in your specific Entra tenant.
- `consumers` — only personal Microsoft accounts.
- `organizations` — only work/school accounts.

For our use case (`vbraju@hotmail.com` signing in to demo the app), set **`AZURE_AD_TENANT_ID=common`**. This, combined with the multitenant + personal registration in step 4.2, is the combination that makes Hotmail logins work.

4.6 Add the Railway callback URL (after deploying)

You will come back here after you deploy to Railway in Section 6:

1. Entra → your app → **Authentication** → **+ Add URI** under the Web platform.
2. Enter `https://YOUR-RAILWAY-DOMAIN/api/auth/callback/microsoft-entra-id`.
3. **Save**.

---

5. Push the code to GitHub

5.1 Create an empty repo on GitHub

1. Open https://github.com/new.
2. Sign in if needed.
3. Repository name: `trustscaleai-app` (or anything you like).
4. Visibility: **Private** is fine; **Public** also works.
5. Do not tick “Add a README”, “Add .gitignore”, or “Choose a license” — the zip already contains all three. An empty repo is what you want.
6. Click **Create repository**. GitHub will show a “quick setup” page with a repo URL like `https://github.com/<your-username>/trustscaleai-app.git` — copy it.

5.2 Push from your local machine

In the unzipped `trustscaleai-app` folder:

```
git init
git add .
git commit -m "Initial TrustScaleAI companion"
git branch -M main
git remote add origin https://github.com/<your-username>/trustscaleai-app.git
git push -u origin main
```

If Git asks for a username and password and a password fails, create a **Personal Access Token** at https://github.com/settings/tokens (fine-grained or classic, with `repo` scope) and use that as the password.

5.3 Verify the push

Refresh the GitHub repo page in your browser — you should see all 55 files including `README.md`, `package.json`, `prisma/schema.prisma`, and `public/mockup.html`. If `public/mockup.html` is missing, your zip extraction may have skipped it — extract again and re-push.

---

6. Deploy to Railway

6.1 Create the Railway project

1. Go to https://railway.app/new.
2. Sign in (use "Continue with GitHub" if you haven't linked your account).
3. Choose **Deploy from GitHub repo**.
4. The first time, you will be asked to grant Railway access to your GitHub account. Grant access to either all repos or specifically to `trustscaleai-app`.
5. Select `trustscaleai-app` from the repo list. Railway begins a deployment immediately using the Nixpacks config in the zip (`nixpacks.toml`).
6. The first build will **fail** at the migration step because `DATABASE_URL` isn't set yet. This is expected. Do not panic.

6.2 Add the PostgreSQL plugin

1. Inside your Railway project, click **+ New** (top right) → **Database** → **Add PostgreSQL**.
2. Railway provisions a Postgres 16 instance in a few seconds.
3. Click on the **Postgres** service tile — you'll see generated credentials. You do not need to copy them.
4. Railway automatically adds a reference variable `${{Postgres.DATABASE_URL}}` across services in the same project. We still need to wire it to your app service (next step).

6.3 Set environment variables on your app service

1. Click the **trustscaleai-app** service tile (not the Postgres one).
2. Go to the **Variables** tab.
3. Click **+ New Variable** (or **Raw Editor** if you prefer paste-all-at-once) and add the following:

Variable	Value
`DATABASE_URL`	`${{Postgres.DATABASE_URL}}` — Railway auto-completes this reference as you type
`AUTH_SECRET`	a 32-byte base64 string (see 6.3.1 below)
`AUTH_TRUST_HOST`	`true`
`NEXTAUTH_URL`	filled in at step 6.4 — leave empty for now
`GOOGLE_CLIENT_ID`	from Section 3.4

Variable	Value
`GOOGLE_CLIENT_SECRET`	from Section 3.4
`AZURE_AD_CLIENT_ID`	from Section 4.2 (Application (client) ID)
`AZURE_AD_CLIENT_SECRET`	from Section 4.3 (the **Value** you copied)
`AZURE_AD_TENANT_ID`	`common`

6.3.1 How to generate AUTH_SECRET On your local terminal:

```
openssl rand -base64 32
```

This prints a 44-character random string like `k7x2p...` — paste that as `AUTH_SECRET`. If you do not have openssl, you can use Node:

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
```

6.4 Generate the Railway domain

1. On your app service → **Settings** → scroll to **Networking** → **Generate Domain**.
2. Railway creates a domain like `trustscaleai-app-production.up.railway.app`. Copy it.
3. Go back to **Variables** and set `NEXTAUTH_URL` to `https://trustscaleai-app-production.up.railway.app` (your exact domain, with `https://`, no trailing slash).

6.5 Back-fill the OAuth redirect URIs

Now that you have a real Railway domain, update both OAuth providers:

- **Google Cloud Console** → APIs & Services → Credentials → your OAuth client → **Edit** → **Authorized redirect URIs** → add `https://<your-railway-domain>/api/auth/callback/google` → **Save**.
- **Microsoft Entra** → your app registration → **Authentication** → **+ Add URI** under Web → add `https://<your-railway-domain>/api/auth/callback/microsoft-entra-id` → **Save**.

6.6 Redeploy and run migrations

Railway redeploys automatically any time you change a variable. Watch the **Deployments** tab:

1. The build phase runs `npm ci && npm run build`.
2. The start phase runs `npm run db:migrate && npm run start`.
3. On a fresh database, `prisma migrate deploy` has no committed migrations to run yet (the zip ships with `schema.prisma` but no `prisma/migrations/` folder). To create the tables the very first time, you have two options:

   Option A — quick-start (recommended for demos). Temporarily change the Railway start command to push the schema directly:

   - App service → **Settings** → **Deploy** → **Custom Start Command** → set to:

     ```
     npx prisma db push --skip-generate --accept-data-loss && npm run start
     ```

   - Save, wait for the next deploy to finish, then change it back to the default:

```
npm run db:migrate && npm run start
```

Option B — proper migrations (recommended for production). Run Prisma migrations locally against a dev database, commit the generated `prisma/migrations/` folder, and push to GitHub. Railway's next deploy will then apply them. Steps:

```
# locally, with DATABASE_URL pointing at a dev database
npx prisma migrate dev --name init
git add prisma/migrations
git commit -m "Initial migration"
git push
```

6.7 Test the deployed app end-to-end

1. Open `https://<your-railway-domain>` in a browser.
2. You should be redirected to `/signin`.
3. Click **Continue with Microsoft**:
   - Sign in as `vbraju@hotmail.com`
   - Approve the permission consent screen (first time only)
   - You should land on `/dashboard` with an empty engagement list
4. Click **Sign out** (top right), then try **Continue with Google**:
   - Sign in as `vbraju@gmail.com`
   - Approve the consent screen
   - You should land on `/dashboard` — now signed in as the Gmail identity
5. Click **+ New engagement** → fill the form (code like `ENG-2026-DEMO-001`, client `Acme Demo`) → **Create engagement**.
6. You land on the engagement detail page.
7. Fill **Add stakeholder** (name, email, organization, role) → **Add stakeholder** → the stakeholder row appears in the table.
8. Click the **Open full product mockup** button — a new tab opens with the full TrustScaleAI mockup (the static HTML).

If all eight steps work, deployment is complete.

---

7. Running the app locally for development

If you want to iterate on the code before pushing to production, a local environment is the best place.

7.1 Start a local Postgres

Easiest path is Docker Desktop:

```
docker run --name tsai-pg \
  -e POSTGRES_PASSWORD=pass \
  -e POSTGRES_DB=tsai \
  -p 5432:5432 \
  -d postgres:16
```

Or install Postgres natively: https://www.postgresql.org/download. Once running, the connection string is `postgresql://postgres:pass@localhost:5432/tsai?schema=public`.

7.2 Fill the local `.env`

Edit `.env` in the project root:

```
DATABASE_URL="postgresql://postgres:pass@localhost:5432/tsai?schema=public"
AUTH_SECRET="<openssl rand -base64 32 output>"
NEXTAUTH_URL="http://localhost:3000"
AUTH_TRUST_HOST="true"

GOOGLE_CLIENT_ID="<from Google Cloud>"
GOOGLE_CLIENT_SECRET="<from Google Cloud>"

AZURE_AD_CLIENT_ID="<from Entra>"
AZURE_AD_CLIENT_SECRET="<from Entra>"
AZURE_AD_TENANT_ID="common"
```

7.3 Apply the schema

```
npm run db:push
```

This creates all the tables (`User`, `Account`, `Session`, `Engagement`, `Stakeholder`, etc.) without needing a migrations folder.

7.4 (Optional) seed demo engagements

```
npm run db:seed
```

This inserts two demo engagements (Regio Bank + NordPharma Group) so the dashboard has visible data before you create your own.

7.5 Run the dev server

```
npm run dev
```

- Open http://localhost:3000 → redirect to `/signin`
- Sign in with `vbraju@gmail.com` or `vbraju@hotmail.com`
- Create an engagement, add stakeholders, click through the mockup

Hot reload is enabled — save a file and the page refreshes.

---

8. Repository layout

```
trustscaleai-app/
├── prisma/
│   ├── schema.prisma         # User/Account/Session + Engagement + Stakeholder
│   └── seed.ts               # Optional demo data (npm run db:seed)
├── public/
│   └── mockup.html          # Full TrustScaleAI mockup (served at /mockup.html)
├── src/
│   ├── app/
│   │   ├── layout.tsx          # Root MUI theme provider
│   │   ├── page.tsx            # Redirects to /signin or /dashboard
│   │   ├── globals.css
│   │   ├── signin/page.tsx     # Public sign-in page with SSO buttons
│   │   ├── dashboard/page.tsx  # Auth-protected engagement list
```

```
│   │   ├── engagements/
│   │   │   ├── new/page.tsx    # Create engagement (Server Action)
│   │   │   └── [id]/page.tsx   # Detail + stakeholder list + add form
│   │   ├── mockup/page.tsx     # Redirects to /mockup.html
│   │   └── api/
│   │       ├── auth/[...nextauth]/route.ts   # Auth.js route handler
│   │       └── engagements/                  # REST endpoints
│   ├── components/
│   │   ├── AppShell.tsx        # Left nav + topbar + sign-out
│   │   ├── EngagementForm.tsx  # Create form (client, useFormState)
│   │   └── StakeholderForm.tsx # Add-stakeholder form
│   ├── lib/
│   │   ├── auth.ts             # Auth.js config (Google + Entra providers)
│   │   ├── auth-handlers.ts    # Route shim for the [...nextauth] handler
│   │   ├── prisma.ts           # Prisma singleton with hot-reload guard
│   │   ├── theme.ts            # MUI theme (mockup palette)
│   │   └── actions.ts        # Server actions: createEngagement, addStakeholder
│   ├── middleware.ts         # Auth gate — redirects non-public routes to /signin
│   └── types/next-auth.d.ts    # Session.user.id / role augmentation
├── .env.example                # Template for local and Railway env vars
├── .gitignore
├── next.config.js
├── nixpacks.toml             # Railway build spec (Node 20, npm ci, npm run build)
├── railway.json              # Railway deploy spec (start command + health check)
├── package.json
├── tsconfig.json
└── README.md                   # This file
```

---

9. REST API

All endpoints require an authenticated session cookie (get one by signing in first — then the browser's cookie authenticates subsequent fetches).

Method	Path	Request body	Response
GET	`/api/engagements`	—	`Engagement[]` with stakeholder counts
POST	`/api/engagements`	`{ code, client, industry?, variant?, phase?, phaseName?, status?, progress?, sponsor?, delivery?, nextGate?, startDate?, targetDate?, region?, notes? }`	Created engagement, HTTP 201

Method	Path	Request body	Response
GET	`/api/engagements/:id`	—	Engagement + stakeholders
GET	`/api/engagements/:id/stakeholders`	—	`Stakeholder[]`
POST	`/api/engagements/:id/stakeholders`	`{ fullName, email, organization, role, side?, influence?, interest?, phone?, notes? }`	Created stakeholder, HTTP 201

Error responses:

- `401 Unauthorized` — no valid session
- `400 Bad Request` — Zod validation failure (body contains `{ error: <flattened issues> }`)
- `404 Not Found` — engagement id does not exist
- `409 Conflict` — engagement `code` already used

---

10. Troubleshooting

Symptom	Root cause and fix
`redirect_uri_mismatch` on Google	The URL you are being redirected to is not in the Google OAuth client's **Authorized redirect URIs** list. Open the client in Google Cloud Console → add `https://<railway-domain>/api/auth/callback/google` (and `http://localhost:3000/api/auth/callback/google` for local). Save, wait 1 minute for propagation.
`AADSTS50020: User account from identity provider does not exist in tenant` on Microsoft sign-in	The Entra app is single-tenant or does not permit personal accounts. Go to App registration → **Authentication** → change **Supported account types** to **Accounts in any organizational directory and personal Microsoft accounts**. Save. Also confirm `AZURE_AD_TENANT_ID=common` in Railway variables.
`AADSTS700016: Application with identifier '...' was not found`	`AZURE_AD_CLIENT_ID` does not match the app registration's Application (client) ID. Re-copy it from the Entra app's **Overview** page.

Symptom	Root cause and fix
`invalid_client` from Microsoft	The client secret is wrong. You either copied the **Secret ID** instead of the **Value**, or the secret has expired. Create a new client secret in **Certificates & secrets**, copy the **Value**, paste into `AZURE_AD_CLIENT_SECRET` in Railway.
`access_denied` from Google while in Testing mode	The Gmail address you are signing in with is not on the **Test users** list. Add it at Google Cloud Console → OAuth consent screen → Test users.
`PrismaClientInitializationError` on Railway	`DATABASE_URL` is missing or wrong. Verify in **Variables** that it equals `${{Postgres.DATABASE_URL}}` (Railway shows this as a blue reference pill). Restart the service.
`relation "User" does not exist` (or similar)	The database schema has not been applied. Use Option A in Section 6.6 to run `prisma db push` once, or follow Option B with committed migrations.
Sign-in succeeds but you're stuck in a redirect loop	Either `AUTH_TRUST_HOST` is not `true`, or `NEXTAUTH_URL` does not start with `https://`. Both are required behind Railway's proxy.
"Callback URL mismatch" after changing the Railway domain	Railway regenerates domains sometimes; anywhere you previously pasted `https://<old-domain>/...` needs updating in Google and Microsoft. Easiest fix: set a **custom domain** in Railway → **Settings** → **Networking**, then that URL never changes.

---

11. Security notes

- Secrets live only in `.env` (git-ignored) locally and in **Railway variables** in production. Never commit `.env`.
- `AUTH_SECRET` must be at least 32 bytes of entropy. Regenerate it if it has ever been exposed.
- Session cookies are HTTP-only, Secure (in production), and SameSite=Lax.
- All API routes call `await auth()` before touching the database.
- Auth.js handles CSRF tokens on sign-in and sign-out flows.
- Prisma parameterises all queries — no SQL injection possible through the provided server actions or REST endpoints.

---

12. Uninstall / teardown

If you're done with the demo and want to stop being billed:

1. **Railway** → project → **Settings** → **Danger zone** → **Delete project**. This deletes the service and the Postgres database.
2. **Google Cloud** → project selector → your project → **IAM & Admin** → **Settings** → **Shut Down**.
3. **Microsoft Entra** → **App registrations** → your app → **Delete**. The free tenant itself is not billed, so you can leave it provisioned for future projects.
